TokaDB: A Unified Storage Engine for Training-Serving Data Management in Large Recommendation Models

Peng Fang*
Huazhong University of Science and Technology
Wuhan, China
fangpeng@hust.edu.cn

Kelei Guo*
Huazhong University of Science and Technology
Wuhan, China
guokelei@hust.edu.cn

Cheng Chen†
ByteDance Inc.
Beijing, China
chencheng.sg@bytedance.com

Wei Zhang
ByteDance Inc.
Beijing, China
zhangwei.95@bytedance.com

Mingming Chen
Huazhong University of Science and Technology
Wuhan, China
chenmingming@hust.edu.cn

Huaye Xu
ByteDance Inc.
Beijing, China
xuhuaye@bytedance.com

Borong Meng
ByteDance Inc.
Beijing, China
mengborong@bytedance.com

Zongjia Chen
ByteDance Inc.
Beijing, China
chenzongjia@bytedance.com

Mingshuai Wang
ByteDance Inc.
Beijing, China
wangmingshuai.1205@bytedance.com

Luping Wang
ByteDance Inc.
Beijing, China
wangluping.502@bytedance.com

Yuan Zhang
ByteDance Inc.
Beijing, China
zhangyuan.42@bytedance.com

Shiru Ren
ByteDance Inc.
Beijing, China
renshiru2000@gmail.com

Fang Wang†
Huazhong University of Science and Technology
Wuhan, China
wangfang@hust.edu.cn

Dan Feng
Huazhong University of Science and Technology
Wuhan, China
dfeng@hust.edu.cn

Abstract

Large recommendation models (LRM), powered by Transformer architectures, are rapidly becoming the backbone of modern recommendation systems. However, their training-serving lifecycle imposes unprecedented challenges on data management: hybrid access patterns for user behavior sequences, cost-performance tradeoffs for exabyte-scale KV Cache data, and resource contention between prioritized workloads during training and serving. Existing approaches fail to address these challenges simultaneously, leading to performance bottlenecks or excessive costs. In this paper, we present TokaDB, a unified storage engine for LRM that efficiently manages both user behavior sequences and KV Cache data across training and serving stages. TokaDB introduces three core innovations: (1) *Hybrid-Access Specialized Data Organization* that segments sequence data by time and decouples indexes from data to support both high-throughput point reads and high-bandwidth range scans; (2) *Cost-Efficient Cooperative I/O Engine* that co-optimizes the read and write paths to reduce tail latency while sustaining high throughput; (3) *Priority-Based Resource Manager* that enforces priority-aware sharing of CPU, cache, and I/O resources to eliminate prioritized workload interference and ensure predictable performance. TokaDB has been deployed across production services at ByteDance and validated under real LRM workloads. Experimental results show that TokaDB achieves 4–10× end-to-end speedups over mainstream systems including HBase, Doris, and Doris+, satisfying all critical performance dimensions required by LRM.

CCS Concepts

• **Information systems** → **Database management system engines**; **Recommender systems**; • **Computing methodologies** → **Machine learning**.

Keywords

LRM, Training and Serving, Data Management, Storage Engine

*Both authors contributed equally to this research.
†Corresponding Author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

SIGMOD Companion '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2450-3/2026/05
https://doi.org/10.1145/3788853.3803078

ACM Reference Format:
Peng Fang, Kelei Guo, Cheng Chen, Wei Zhang, Mingming Chen, Huaye Xu, Borong Meng, Zongjia Chen, Mingshuai Wang, Luping Wang, Yuan Zhang, Shiru Ren, Fang Wang, and Dan Feng. 2026. TokaDB: A Unified Storage Engine for Training-Serving Data Management in Large Recommendation Models. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26), May 31-June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3788853.3803078

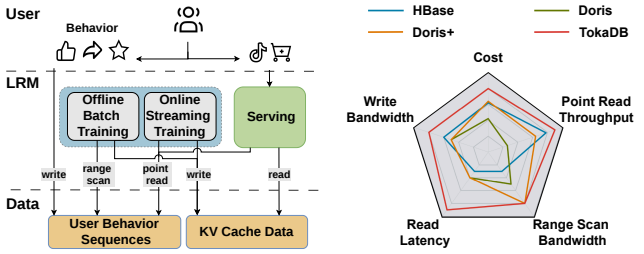
1 INTRODUCTION

As large language models (LLMs) continue to advance rapidly [13, 57], they are increasingly reshaping recommendation modeling as well. Recent research proposes large recommendation models (**LRM**) [52, 61], which apply Transformer architectures [45] to recommendation tasks and have quickly become a central focus in modern recommendation systems [48]. Unlike traditional recommendation models [12, 38], the LRM leverages the scalability of Transformers and their ability to capture long-term user behavior patterns, demonstrating clear advantages in exploring scaling laws in recommendations [5, 53, 54]. These advancements have already translated into large-scale production deployments: Meta has deployed a trillion-parameter LRM for generative recommendation tasks [52], while ByteDance has extensively applied LRM across core e-commerce [11] and advertising platforms [14].

Industry-scale LRM systems, including those at ByteDance, typically structure the model lifecycle into two primary stages: **Training** and **Serving** [11, 19, 23, 44]. As shown in Figure 1(a), the training stage adopts a dual-workflow design to handle the temporal variability in user preferences [41]. Offline batch training [30] exploits large-scale historical data to establish a robust model foundation, while online streaming training [50] continuously incorporates recent user interactions to capture rapidly shifting interests. In parallel, the serving process performs inference and uses KV Cache techniques [27] to speed up long-sequence processing, which enables low-latency, personalized recommendations for end users. Overall, the two-stage lifecycle introduces heterogeneous and tightly coupled data access demands across training and serving.

These stages in LRM are closely integrated with two primary categories of data: **User Behavior Sequences** [24, 28] and **KV Cache data** [27, 39]. User behavior sequences capture engagement signals (e.g., likes, reposts, stars). Each user behavior sequence already contains tens of thousands of records, and the overall dataset has grown to tens of petabytes at ByteDance. These sequences serve a dual role [32]: they provide supervision during training to help the model infer user preferences, and serve as contextual input during serving to improve recommendation relevance. KV Cache data functions as a persistent feature cache rather than transient attention KV tensors used in general inference. Because each user's serving context derives from a stable behavior sequence, KV Cache entries are proactively precomputed after offline training and incrementally refreshed during online updates [11], significantly reducing repeated computation and serving overhead. In ByteDance's production deployment, hundreds of LRM variants run in parallel, each maintaining its own KV Cache data, with the aggregate volume reaching the exabyte scale.

As LRM continues to evolve, its overall efficiency increasingly depends on the database system's ability to manage these two data modalities. Since both data modalities are jointly accessed across the training and serving stages, unified management is indispensable for reducing operational complexity and improving overall resource utilization. However, mainstream data engines, such as HBase [47] and Doris [4], have been unable to support LRM effectively due to the following key challenges:



(a) LRM Workflow (b) Multi-Dimensional Comparison

Figure 1: The LRM workflow and the objective of TokaDB.

(1) Hybrid Access Mismatch for User Behavior Sequences. In ByteDance, online streaming training and serving require high-throughput point reads (up to 6.5 million QPS), while offline batch training demands continuous high-bandwidth range scans (over 2.75 TB/s), making hybrid access particularly challenging. Row-oriented stores (e.g., HBase) offer strong point read performance but provide only about 1/4 of the required range scan bandwidth. Column-oriented stores (e.g., Doris) achieve efficient range scans but exhibit point read performance up to 6× below production requirements. Doris+, an enhanced version of Doris with point read optimizations, improves performance by up to 4×, but still fails to meet system requirements. **(2) Cost-Performance Barrier for KV Cache Data.** With KV Cache data reaching the exabyte scale at ByteDance, our production cost model shows that the necessary SSD capacity would cost roughly 70% of a GPU cluster equipped with NVIDIA A100 GPUs, making SSD-based storage economically impractical. However, inherent HDD characteristics, including slow replicas [34], high cross-block random access latency, and limited bandwidth, prevent the database from meeting the 100 ms 99th-percentile (P99) read latency target for LRM serving and the sustained write bandwidth demands of LRM training. In our measurements, HBase and Doris on HDD-based HDFS exhibit P99 read latency up to 4× worse than the 100 ms target and provide under 40% of the required write bandwidth. **(3) Resource Contention Between Prioritized Workloads.** In production at ByteDance, high-priority workloads (e.g., operations during LRM serving) share limited CPU cycles, cache capacity, and storage bandwidth with low-priority workloads such as operations during LRM training and database background activities. In existing systems, these low-priority tasks can aggressively consume shared resources and directly interfere with high-priority serving operations. Our empirical measurements show that point read throughput on user behavior sequences drops by 60% and P99 read latency on KV Cache data increases by 120% under contention. This priority interference makes it difficult for existing general-purpose storage engines to deliver predictable performance for LRM workloads.

Overview of TokaDB. We propose TokaDB, a unified storage engine designed to manage both user behavior sequences and KV Cache data efficiently across the training and serving stages in LRM.

To resolve the conflicting access requirements of user behavior sequences, TokaDB introduces the *Hybrid-Access Specialized Data Organization*. It first employs a *Detached-Segment Index Layout* that preserves per-user physical locality for fast point reads while partitioning user behavior records into time-based segments to sustain high-bandwidth range scans. TokaDB further restores spatial locality for fragmented segments via *Locality-Triggered Garbage Collection* and reduces storage costs through *SSD/HDD Hot-Cold Tiering*. **To address KV Cache's cost-performance barrier**, TokaDB implements the *Cost-Efficient Cooperative I/O Engine*, which optimizes multiple stages of the I/O path. For reads, it applies two techniques to maintain low P99 read latency under high throughput: *Adaptive Backup Reads* to mitigate performance jitter caused by slow replicas, and *Hardware-Aware Block Alignment* to eliminate cross-block seeks. For writes, it employs a *Log as Data* approach to provide stable write bandwidth for LRM workloads. **To support both training and serving without resource contention**, TokaDB introduces the *Priority-Based Resource Manager*. It coordinates high-priority LRM serving workloads against low-priority LRM training workloads and database background activities by using a *Preemptive Thread Scheduler* and a *Hierarchical Cache Allocator*. It further mitigates storage-bandwidth contention through a *Multi-Layer Integrated Rate Limiter*, which also shields high-priority serving workloads from bursty low-priority traffic. Figure 1(b) illustrates TokaDB's design objective of meeting the multi-dimensional requirements of LRM, enabling unified support for both training and serving.

Contributions. The main contributions of this paper are:

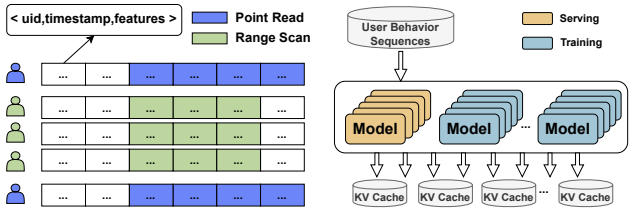
- A systematic characterization of LRM workloads reveals the core challenges of hybrid access, cost-performance imbalance, and prioritized workloads interference, motivating a specialized storage engine for LRM.
- The design and implementation of TokaDB, a unified storage engine for training-serving data management in LRM. TokaDB has been deployed in large-scale production environments at ByteDance. To our knowledge, it is the first system that unifies the management of user behavior sequences and KV Cache data across both training and serving stages.
- An extensive evaluation of large-scale, real-world LRM workloads from ByteDance. The results demonstrate that TokaDB achieves a 4–10× end-to-end performance improvement over popular solutions, and consistently meets all performance requirements of LRM workloads.

2 BACKGROUND

LRM has become a core paradigm for modern recommendation systems [52, 61]. Its workflow is tightly coupled with two data modalities: user behavior sequences and KV Cache data. As these data grow, the performance and cost of storing them increasingly limit the deployment of LRM. To understand these bottlenecks, we first classify LRM data engines by their storage layouts and then analyze the workload characteristics of these two modalities.

2.1 Storage Layouts in Data Engines for LRM

We classify data engines for LRM by their physical storage layout into two broad categories: row stores and column stores, following the established taxonomy in prior literature [1, 2, 6]. Row stores



(a) User Behavior Sequences (b) KV Cache Data

Figure 2: Two primary workloads in LRM.

persist tuples in a row-wise manner as key-value records, whereas column stores organize data in a column-oriented layout within partitioned files. This fundamental distinction in storage layout dictates how systems buffer, persist, and structure data on disk.

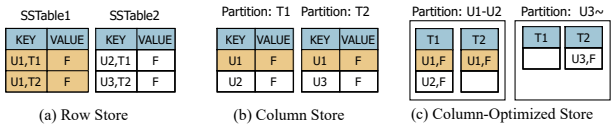
Row Stores. Row-oriented database systems [8, 29, 47] typically employ a hierarchical LSM-tree-based [37] storage architecture. Write operations are first accumulated in an in-memory structure and then periodically flushed to immutable sorted files (e.g., SSTables). Background compaction merges these files into larger sorted files, maintaining a globally ordered key space at the granularity of individual key-value entries.

Column Stores. In contrast, column-oriented database systems [4, 40, 43] store data in columnar files, typically using formats such as Parquet [46]. Within each columnar file, data is organized into column groups, and each column is stored contiguously inside a group, which maximizes range scan bandwidth and compression effectiveness. This columnar layout allows queries to read only the columns relevant to their predicates, enabling efficient filtering and reducing I/O for analytical workloads.

2.2 Workload Analysis for LRM

User Behavior Sequences. User interactions across multiple products (e.g., likes, reposts, stars) are captured as structured records and stored in database [60]. Sorting each user's actions chronologically creates their historical behavior sequences. At ByteDance, a single user's behavior sequence can contain hundreds of thousands of records over two years, and the collective data for all users can reach tens of petabytes. By using these sequences for both training and serving, recommendation models can effectively capture a user's long-term interests and short-term preference shifts [24, 28]. Each interaction is stored as $\langle uid, timestamp, features \rangle$, uid uniquely identifies the user, $timestamp$ records the exact timestamp to ensure temporal ordering, and $features$ is a compact serialized payload containing multi-dimensional interaction details. A typical 64-byte $features$ might include 3 int64 integers (24 bytes) for identifiers, and 4 strings (40 bytes) for scenario labels and context tags.

As shown in Figure 2(a), LRM workloads define two core access patterns. For serving and online streaming training, *Point Read* retrieves the behavior sequence of a specified user, typically returning thousands to hundreds of thousands of records (depending on LRM scale), and queries per second (QPS) is the dominant performance metric. For offline batch training, *Range Scan* retrieves behavior data for all users within a specified range, applying the same per-user record limit as point read; in this case, high bulk bandwidth is the primary objective, and higher latency is acceptable.



(a) Row Store (b) Column Store (c) Column-Optimized Store

Figure 3: Storage structures for user behavior sequences.

KV Cache Data. KV Cache data stores the intermediate Key and Value matrices of the attention mechanism [45], allowing models to reduce GPU computation and inference latency [7]. At ByteDance, only a few models are used for serving, while a large number of models are continuously trained to explore new architectures and techniques, as shown in Figure 2(b). As a result, hundreds of LRM variants run in parallel. They share the same user behavior sequences, but each model has its own KV Cache data, pushing the total KV Cache data beyond the exabyte scale.

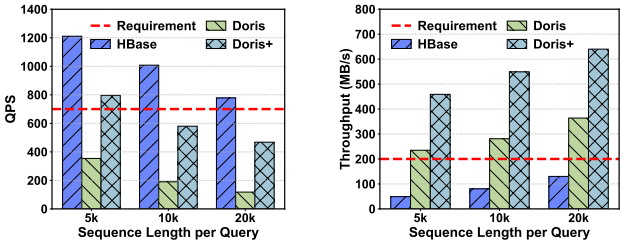
KV Cache data is organized hierarchically: a small, high-speed tier (HBM/DRAM) near the model retains hot entries, while the exabyte-scale full data resides in a backend database. This means that requests reaching the backend have a low cache hit rate. At ByteDance production scale, KV Cache data is stored persistently as compressed 1-2 MB entries. Since each user's serving context is determined by a stable behavior sequence, KV Cache entries are precomputed for hot users after offline training and are incrementally updated online. This avoids repeated computation and cold-start overhead, reducing serving latency and computation costs. For serving workloads, predictable latency is critical; even a small fraction of long-tail delays (e.g., >100 ms) can significantly degrade the user experience. As a result, the backend database must simultaneously provide (1) low P99 read latency with sufficient throughput for serving and (2) sustained high-bandwidth writes for training.

3 MOTIVATION

To explore the bottlenecks of LRM storage processes, we performed three motivating experiments that expose fundamental limits of existing databases in this scenario. Specifically, we: (1) evaluated the performance trade-offs of row, column, and column-optimized stores under user behavior sequences' hybrid reads; (2) examined HDD-based database performance under KV Cache data's cost-performance constraints; and (3) explored cross-workload interference when high-priority serving workloads and low-priority training/background workloads run concurrently. All experiments use the same settings: two compute nodes (64 CPU cores, 512 GB memory each) and HDFS backed by four 7 TB SSDs and eight 18 TB HDDs. HDFS is provisioned with sufficient CPU and memory to avoid becoming a bottleneck. This setup reflects typical production-scale database deployments at ByteDance.

3.1 Hybrid Access Mismatch for User Behavior Sequences

User behavior sequences serve as the foundation for both training and serving, and are primarily accessed via *Point Read* and *Range Scan*. As shown in Figure 3, existing databases fall into three categories: (1) In **row stores**, user behaviors are stored as individual key-value records with composite keys such as $\langle uid, timestamp \rangle$. Thus, all records of a given user are laid out contiguously on SSTables and can be fetched with a single sequential read; however, the



(a) Point Read Throughput (b) Range Scan Bandwidth

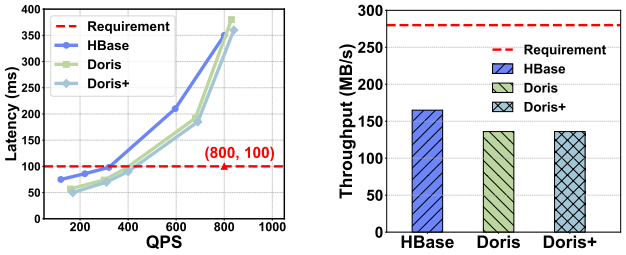
Figure 4: Performance on user behavior sequences.

system cannot precisely filter by time. (2) In **column stores**, user behaviors are stored in time-partitioned columnar files, enabling efficient time-range filtering; however, each user's sequence is spread across multiple partitions. The model, however, expects a contiguous behavior sequence as input, so it must perform cross-partition joins to assemble the user's data. (3) In **column-optimized stores**, user behaviors are stored in user-partitioned layouts and, within each partition, further organized into time-based column groups; this design also provides efficient time filtering and avoids cross-partition joins. However, fetching a user's full sequence still incurs cross-group join overhead.

To examine the performance trade-offs, we benchmark three representative systems under LRM-aligned workloads: HBase (row store), Doris (column store), and Doris+ (column-optimized store). The dataset contains 2 *million* users, totaling roughly 2.3 *TB*. We measure *Point Read* operations during serving and online streaming training, and *Range Scan* operations during offline training. Figure 4 includes a "requirement" line derived from production statistics in ByteDance, representing the minimum performance needed for LRM workloads at this scale. HBase was the only system to meet the *Point Read* throughput requirement. However, it failed to meet the required *Range Scan* bandwidth, falling up to 4× short of the required performance. HBase stores each user's behavior sequence contiguously . As a result, even for a small time window, the range scan iterator must traverse the user's entire history to return the requested records, causing severe read amplification. In contrast, Doris met the *Range Scan* requirement, but its high cross-partition join overhead made its *Point Read* throughput 72%-85% lower than HBase's and even up to 6× below the requirement. Doris+ achieved notable gains over Doris, improving *Point Read* throughput by 2.3×-3.9× and *Range Scan* bandwidth by 1.7×-1.9×. Although Doris+ eliminated the high cross-partition join overhead of Doris, its physical layout still required cross-column joins, which prevented it from achieving high *Point Read* throughput.

3.2 Cost-Performance Barrier for KV Cache Data

Storing KV Cache data on SSDs is economically impractical at ByteDance production scale, where the total volume has reached the exabyte level. Our production cost model shows that the required SSD capacity alone would cost approximately 70% of an NVIDIA A100 GPU cluster. Deploying KV Cache on HDDs becomes the the most practical option, yet it introduces significant performance constraints. To evaluate the cost-performance limitations of HDD-based deployments for KV Cache data, we conduct experiments



(a) P99 Read Latency vs. QPS (b) Max Write Bandwidth

Figure 5: Performance on KV Cache data.

on a dataset containing 20 *million* KV Cache entries. Each entry is approximately 1.6 MB, totaling 30 TB for a single-replica dataset. The data is stored on HDDs under a three-replica configuration, consistent with production-grade fault tolerance settings. Based on real-world workload statistics, supporting LRM training and serving at this scale requires at least 800 read QPS with P99 read latency under 100 ms, and a minimum write bandwidth of 250 MB/s.

Figure 5 shows that mainstream deployments (HBase, Doris and Doris+ on HDFS) fall well below the required target. Notably, Doris and Doris+ exhibit very similar performance because KV Cache data is stored as a simple key-value structure in both systems. For read, although all systems can reach 800+ QPS, their P99 read latency deteriorates sharply (290 ms for HBase and 350 ms for Doris and Doris+). When constrained to stay within P99 < 100 ms, sustainable throughput drops to only 220 QPS and 300 QPS (27-37% of the required throughput). This is primarily because HDDs provide limited IOPS and perform poorly under random reads. Moreover, HDD performance can temporarily degrade due to transient disk faults, background compaction, or other database background activities. These short-term drops create slow replicas and unstable tail latency. For write, when sustaining an 800 QPS read load, the maximum ingest throughput peaks at only 165 MB/s on HBase and 136 MB/s on Doris and Doris+, far below KV Cache update requirements because most HDD bandwidth is already consumed by read traffic, leaving insufficient capacity for ingestion. The situation is further worsened by the dual-write requirement, where data must be persisted to both the log and the data file for consistency.

3.3 Resource Contention Between Prioritized Workloads

For LRM workloads, serving is the top priority. This affects both user behavior sequences and KV Cache data. For user behavior sequences, *Point Read* during serving has a higher priority than *Range Scan* during training. For KV Cache data, read requests during serving likewise have higher priority than bulk writes during training. However, existing systems cannot effectively allocate resources according to these priorities, leading to resource contention between serving and training. To observe the impact of this contention, we first measure each workload individually to establish baseline performance, and then run high-priority serving workloads and low-priority training workloads concurrently on the same cluster.

Table 1 reports the performance degradation each database experienced under this concurrent deployment. Note that since baseline performance differs across systems, these percentage values cannot

Table 1: Performance degradation with concurrent serving and training workloads.

Metrics	HBase	Doris	Doris+
Point Read Throughput	-65%	-28%	-35%
Range Scan Bandwidth	-18%	-27%	-10%
P99 Read Latency	+40%	+78%	+81%
Write Bandwidth	-50%	-27%	-23%

be used to rank their relative effectiveness, but all databases exhibit severe degradation. Most importantly, two critical metrics for LRM serving, *Point Read Throughput* and *P99 Read Latency*, degrade substantially. HBase's point read throughput drops by 65%, while Doris and Doris+ drop by 28% and 35%, respectively. For P99 read latency, Doris and Doris+ increase by 78% and 81%, while HBase increases by 40%. Further observations show that low-priority training workloads and database background activities generate substantial traffic, consuming most CPU, storage bandwidth, and the shared block cache. As a result, hot data used by high-priority serving workloads is evicted, requests from serving workloads cannot obtain sufficient CPU and I/O resources.

4 DESIGN

Motivated by the bottlenecks identified in §3, we design TokaDB, a unified storage engine for training-serving data management in large recommendation models. We present an overview of TokaDB in §4.1, followed by the three key techniques (§4.2-4.4).

4.1 System Overview

TokaDB manages user behavior sequences and KV Cache data concurrently, supporting both training and serving in LRM. As shown in Figure 6, TokaDB's design is organized into three layers:

- **Hybrid-Access Specialized Data Organization** supports hybrid access on user behavior sequences through three components: *Detached-Segment Index Layout* for physical contiguity and efficient filtering, *Locality-Triggered Garbage Collection* to restore spatial locality for fragmented segments and *SSD/HDD Hot-Cold Tiering* to balance cost and performance (§4.2).
- **Cost-Efficient Cooperative I/O Engine** ensures consistent performance on HDD-based KV Cache data workloads via *Adaptive Backup Reads* to mitigate slow replica tail latency, *Hardware-Aware Block Alignment* to eliminate cross-block seeks, and *Log as Data* to remove dual-write overhead (§4.3).
- **Priority-Based Resource Manager** resolves resource interference via *Preemptive Thread Scheduler* to prevent CPU starvation, *Hierarchical Cache Allocator* to control cache usage, and *Multi-Layer Integrated Rate Limiter* to manage storage bandwidth and protect high-priority workloads from low-priority traffic bursts (§4.4).

Alongside the aforementioned optimizations, TokaDB introduces systematic enhancements: (1) a *Disaggregated Storage-Compute Architecture* with a pluggable storage substrate (ByteStore or HDFS); (2) a *Cluster Manager* providing a global metadata service, a cluster orchestrator, and security & access control; and (3) a *Multi-Interface Support* layer exposing Table, SQL, Vector and other APIs (§5).

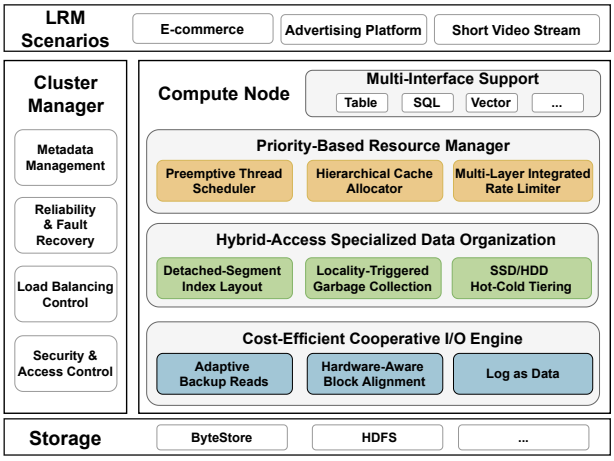


Figure 6: System overview of TokaDB.

4.2 Hybrid-Access Specialized Data Organization

Detached-Segment Index Layout. Row stores suffer from read amplification because they cannot efficiently filter valid data by time, while column stores incur join overhead since they cannot physically organize related records together. TokaDB's *Detached-Segment Index Layout* simultaneously overcomes both limitations. As shown in Figure 7, we implement this layout through two steps. **Step 1** aggregates all behavior records belonging to the same user into a single large key-value (KV) pair. The user ID (e.g., u1) serves as the key, and all corresponding behavior data along with their timestamps (e.g., t_1, t_2, $\cdots$, t_{10000}) are collectively encoded into the value field. Inspired by the KV-separation techniques [8, 22, 33], **Step 2** focuses on optimizing this large value. We first partition the value into time-based segments (e.g., 10 segments for 10,000 records) and store these segments in a dedicated *ValueLog* in order. We then construct a corresponding index for each segment, which includes the segment's address in the *ValueLog*, its time range, and the number of contained records. Finally, we encode all index information of the user's segments into a new value, which is stored in the compact *Index SSTable* (still keyed by the user ID). This transformation results in a lightweight *Index SSTable* and a *ValueLog* that holds the full volume of user behavior data.

For read operations in user behavior sequences, the proposed layout supports efficient data retrieval in both *Point Read* and *Range Scan* operations. In a *Point Read*, we first retrieve the user's index information from the *Index SSTable*. After identifying the target segments, we then fetch their behavioral data from the *ValueLog*. Since the behavior records within segments are time-ordered and stored in an aggregated manner, such reads are highly efficient and avoid the additional join overhead commonly observed in column stores. During a *Range Scan* over a set of users, we first iterate through the KV pairs in the *Index SSTable*. For each specified user within the requested range, similar to *Point Read*, we only need to extract the required segments from the *ValueLog* based on the time range or maximum record count, thus avoiding read amplification caused by fetching full-volume data. For instance, compared to a row store that must scan all 10,000 records to get the latest 1,000

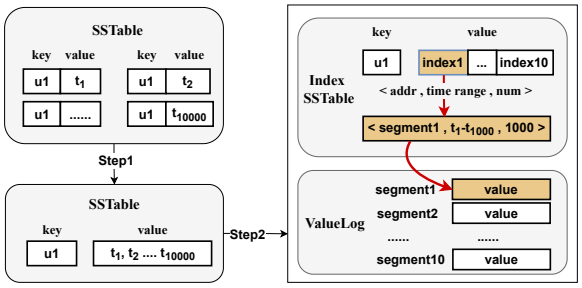


Figure 7: Detached-Segment Index Layout.

behavior records, TokaDB can precisely retrieve only the desired 1,000 records, thereby reducing read amplification by up to 90%.

Locality-Triggered Garbage Collection. Although the *Detached-Segment Index Layout* supports both high-throughput point reads and high-bandwidth range scans, continuous compaction of the *Index SSTable* can cause a user's data segments to become scattered across multiple *ValueLog* files, thereby degrading data locality. Conventional systems, such as BlobDB [22] and TerarkDB [8], perform garbage collection (GC) on the *ValueLog*, but they are designed to reclaim space from expired entries and cannot address user-level fragmentation, thus breaking the data aggregation property required by point read. To solve this, TokaDB implements a *Locality-Triggered Garbage Collection* (LTGC) mechanism that aggregates a user's segments scattered across multiple *ValueLog* files and writes them into a new *ValueLog* file, thereby restoring data locality.

Rather than relying on data expiration, LTGC is driven by the degree of data fragmentation observed during compaction. Specifically, for each key k involved in a compaction, we define a locality score LS_k as the number of distinct *ValueLog* segments containing its data. During compaction, two metrics are computed: (1) the average locality $\bar{LS}$ across all keys, and (2) the fraction F_{LS} of keys whose locality LS_k exceeds a high threshold τ (e.g., $\tau = 5$). LTGC is triggered if either of the following conditions is met:

$$\bar{LS} > \theta \quad \text{or} \quad F_{LS} > \gamma \tag{1}$$

where θ is the average locality threshold (e.g., $\theta = 3$), and γ represents the minimum proportion of highly fragmented keys (e.g., $\gamma = 0.3$). This locality-triggered approach effectively preserves data locality while significantly reducing the background I/O overhead caused by conventional garbage collection.

SSD/HDD Hot-Cold Tiering. Different segments in the *Detached-Segment Index Layout* store user behavior records across distinct time ranges. Most LRM workloads access recent behaviors, while only some large-parameter models read older data (e.g., from two years ago). This access pattern induces hot-cold characteristics: segments with recent data are "hot," whereas older segments are "cold." In production, we observe pronounced skew: the least-active 40% of user-behavior segments account for less than 10% of reads, suggesting substantial tiering potential.

To exploit this skewness, we maintain two types of *ValueLogs*: an SSD-resident one and an HDD-resident one. We introduce a configurable target ratio (F_{HDD}) that denotes the desired fraction of each user's data to reside on HDDs. For example, when ($F_{\text{HDD}} = 0.4$), TokaDB places the oldest (40%) of a user's behavior records on HDD while keeping the most recent (60%) on SSD for low-latency access.

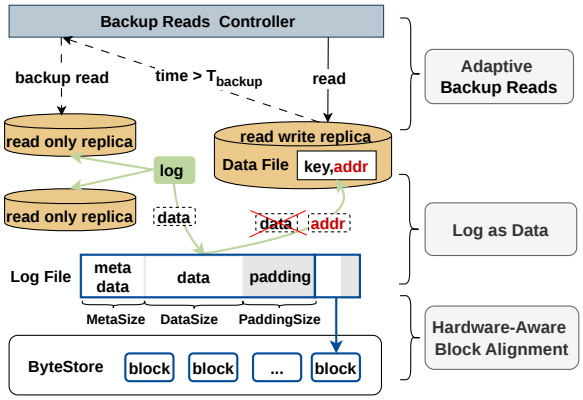


Figure 8: Cost-Efficient Cooperative I/O Engine.

During garbage collection, TokaDB assigns each segment to SSD or HDD by comparing the current per-user distribution with (F_{HDD}) and migrating segments as needed, ensuring that the realized cold-data share tracks the configured target.

4.3 Cost-Efficient Cooperative I/O Engine

Adaptive Backup Reads. As discussed in § 3.2, short-term HDD performance drops create slow replicas and unstable tail latency. To mask such outliers, we introduce a deadline-based *Backup Reads Controller* to the read path. As shown in Figure 8, when a read request to the first replica is not completed within a configurable deadline T_{backup}, the client issues a parallel read to another replica. TokaDB places replicas across different racks, reducing the likelihood of correlated slowdowns at the device level. The first successful response is accepted, and any outstanding requests are immediately canceled to avoid unnecessary resource consumption.

However, the backup read deadline cannot be static. With a fixed T_{backup}, transient congestion increases the fraction of requests exceeding the deadline, which in turn triggers more backup reads and worsens the situation. We therefore introduce a lightweight closed-loop controller driven by a normalized *resource-pressure* signal S_t. Here t denotes the discrete sampling instant, and we sample once every fixed interval $\Delta = 100$ ms; thus $t{+}1$ is exactly the next sampling point 100 ms later. The signal $S_t \in [0, 1]$ is a dimensionless indicator of resource pressure computed over $[t\Delta, (t{+}1)\Delta)$ from device-layer metrics (e.g., queue depth, I/O throughput, device utilization, and request latency). We normalize each metric to $[0, 1]$ and take a weighted aggregate so that $S_t \approx 0$ indicates an idle state and $S_t \approx 1$ indicates saturation. The controller updates the backup deadline by:

$$T_{\text{backup}}(t{+}1) \;=\; T_{\text{backup}}(t) \;+\; \eta\,(S_t - S_0) \tag{2}$$

where $T_{\text{backup}}(t)$ is the deadline applied during sampling step t, η is a tunable parameter controlling the adjustment magnitude (typically set to 20 ms), and S_0 is the target pressure level (typically set to 0.7). When $S_t > S_0$, the controller increases T_{backup} to suppress new backup reads; when $S_t < S_0$, it decreases T_{backup} to resume aggressive tail-latency mitigation. The computation uses only lightweight sampling and arithmetic, introducing $< 1\%$ CPU overhead while stabilizing performance under transient congestion.

Hardware-Aware Block Alignment. Underlying storage systems such as ByteStore manage data using fixed-size physical blocks (typically 2 MB), which serve as the fundamental units for both I/O and caching. While these systems guarantee physical continuity within each block, they do not ensure that logically adjacent blocks are physically contiguous. In a production ByteStore deployment , we measured severe fragmentation: more than 90% of logically adjacent block pairs were physically non-contiguous. This fragmentation poses a critical challenge for KV Cache data: a large value, due to misalignment, can easily span multiple blocks, causing a single logical read to devolve into multiple discrete I/O seeks on HDDs. In our production profiling, such fragmentation increased the average seek count per read by up to 3×, directly inflating tail latency and reducing effective throughput by 29%. Such fragmentation makes low-latency access requirements virtually unattainable.

To eliminate cross-block read amplification at its source, we adopt a *Hardware-Aware Block Alignment* strategy, as illustrated in Figure 8. Specifically, each physical block is treated as a self-contained unit aligned exactly with the hardware block boundary. Within a block, the engine first places the metadata and the data, and then fills the remaining space with padding bytes to ensure full alignment. This guarantees that every KV Cache entry is stored within a single sequential block, completely avoiding cross-block seeks. The padding size is computed as:

$$PaddingSize = BlockSize - \big(MetaSize + DataSize\big) \tag{3}$$

On the write path, this alignment policy ensures that each KV Cache entry is mapped one-to-one onto a single physical block. Consequently, every subsequent read can be fulfilled by a single block-sized I/O, completely eliminating cross-block seeks. This design represents a deliberate trade-off: it introduces a small and bounded spatial amplification (the "padding" shown in Figure 8) in exchange for predictable, single-shot accesses and consistently low P99 read latency in HDD-based KV Cache data workloads (§6.4).

Log as Data. Conventional databases rely on two complementary mechanisms for data protection: write-ahead logs to ensure crash-consistency and data replication for fault tolerance against node failures. While this design ensures strong reliability, it also introduces redundant persistence; each write must be flushed twice: first to the log and then again to the main data file (e.g., SSTable). In large-scale LRM workloads, this dual-write pattern wastes valuable bandwidth on HDDs with limited write bandwidth, becoming the dominant bottleneck in the training data pipeline.

TokaDB addresses this bandwidth inefficiency through its *Log as Data* design, illustrated in Figure 8. Instead of maintaining separate storage for the log file and the data file, TokaDB unifies them by treating the log as the primary data store. To realize this, we modify the LSM-tree format so that it no longer stores full value records; instead, each entry maintains a lightweight pointer to the corresponding data block in the log. During reads, the system first retrieves this pointer from the LSM-tree and then fetches the actual data directly from the log. This design removes the redundant second write, eliminating the bandwidth overhead of dual persistence while preserving durability and crash-consistency semantics. In terms of data reliability, TokaDB replicates log data at the storage

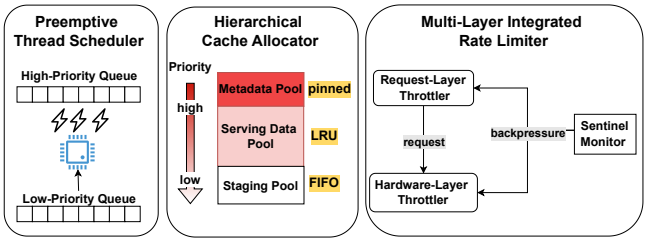


Figure 9: Priority-Based Resource Manager.

layer with the same number of data replicas, thereby preserving crash-consistency and replication durability guarantees.

4.4 Priority-Based Resource Manager

Preemptive Thread Scheduler. TokaDB employs a user-level preemptive thread scheduler to prioritize latency-sensitive serving requests and mitigate CPU starvation, as illustrated in Figure 9. Unlike OS-level preemption, this scheduler operates entirely within user space [25], enabling sub-millisecond context switching among different workloads without kernel intervention. It maintains per-core *High-Priority* (**HPQ**) and *Low-Priority* (**LPQ**) queues: **HPQ** is used to run LRM serving workloads, including point read for user behavior sequences and read requests for KV Cache data, while **LPQ** handles training workloads as well as database background activities (e.g., compaction and garbage collection).

When a high-priority request is admitted, it is inserted into the **HPQ**, and the scheduler always selects from the **HPQ** whenever it is non-empty. If a low-priority request is executing, the scheduler preempts it and dispatches the high-priority request to the CPU. When a high-priority request issues an asynchronous I/O, it yields the CPU but remains in the **HPQ**. Once the I/O completes, the request becomes immediately eligible for scheduling and triggers the preemption of any running low-priority requests, minimizing additional scheduling delays. To prevent low-priority workloads from being indefinitely starved of CPU resources, TokaDB assigns a small number of dedicated CPU cores to the **LPQ**. In practice, only a limited subset of cores (e.g., 8 out of 64) is reserved for low-priority execution, ensuring that these workloads make steady progress without interfering with high-priority serving workloads. TokaDB incurs only negligible CPU overhead while reducing P99 read latency by approximately 25% (shown in § 6.4).

Hierarchical Cache Allocator. In traditional databases, cache budgets are typically allocated at the table granularity [21, 47]: user behavior sequences and KV Cache data are each governed by a single, static cache policy. However, this simple approach is inefficient for LRM workloads. First, in TokaDB's KV-separation engine, locating a value requires accessing file metadata and index SSTables, making it critical to keep these index structures cached to avoid extra I/O and latency. Second, high-priority workloads exhibit greater data locality than low-priority workloads. For user behavior sequences, point read operations repeatedly access hot users, and therefore achieve higher cache locality than range scans. For KV Cache data, although the overall cache hit ratio is low, the heavier read traffic from serving models is still more concentrated than bulk writes during training.

To address these issues, TokaDB adopts a hierarchical, priority-based *Cache Allocator*. At the top, the highest-priority *Metadata Pool* stores filter blocks and index SSTables of LSM-Tree together with other critical metadata. These blocks are touched on almost every lookup but occupy only a small fraction of the total cache budget. TokaDB pre-warms and pins this portion of the cache whenever a compute node starts. This method guarantees a deterministic, single-read lookup path and eliminates the double-read latency problem common in key-value separation [33].

The second tier, the *Serving Data Pool*, is dedicated to LRM serving workloads and therefore receives the majority of the cache budget, typically more than 50% of the total. This tier caches user behavior sequences data accessed by serving-time point read operations as well as KV Cache data for serving models. Entries in this tier are evicted using an LRU policy [36], which has proven effective in our production environment. The remaining cache capacity is assigned to the third tier, the *Staging Pool*. It caches data accessed by LRM training and database background activities. The Staging Pool is managed with a FIFO policy [20], allowing low-priority workloads to exploit short-term locality without causing churn in the Serving Data Pool. Together, this three-tier cache allocator reduces cache pollution and stabilizes hit rates across prioritized workloads, improving throughput by about 63% and reducing P99 read latency by roughly 13%.

Multi-Layer Integrated Rate Limiter. LRM training traffic and database background activities fluctuate over time, and bursts of these low-priority workloads can saturate the device and inflate latency for high-priority serving workloads. Databases typically use a rate limiter to mitigate such contention. However, in LRM workloads, a single-layer rate limiter is insufficient. A request-layer limiter cannot guarantee stable physical I/O load, because even the same type of requests (e.g., range scans over different ranges) can generate very different amounts of I/O. A purely physical-layer limiter throttles I/O blindly and may block high-priority workloads together with low-priority workloads.

As shown in Figure 9, TokaDB employs a multi-layer rate limiter combining a *Request-Layer Throttler* with a *Hardware-Layer Throttler*. Each request first acquires tokens from a priority-based *Request-Layer Throttler* (serving workloads > training workloads/background activities). At the hardware I/O layer, each I/O request must acquire tokens from a *Hardware-Layer Throttler*, whose token rate is dynamically adjusted according to the current state of the underlying storage system. A *Sentinel Monitor* tracks device utilization and queue depth. When either exceeds a high watermark, it issues back-pressure by reducing token generation for low-priority queues and gradually restores their rate once the system falls below a low watermark. By integrating multi-layer throttling with real-time feedback, this design prevents low-priority bursts from monopolizing storage bandwidth and ensures predictable latency for high-priority serving workloads, as shown in §6.4.

5 Implementation

TokaDB has been widely adopted across a variety of production services at ByteDance. To support these large-scale deployments, TokaDB adopts a **disaggregated storage-compute architecture**, as shown in Figure 6. Compute nodes handle resource management,

Table 2: Statistics of evaluated workloads

Metric	E-commerce	Advertising
User Behavior Sequences		
Dataset Total Size	24.52 TB	37.11 TB
Average length	15.32k	24.28k
KV Cache Data		
KV Entry Size	1.6 MB	1.6 MB
Dataset Total Size	122 TB	167 TB

data organization, and interactions with the underlying storage tier. On the storage side, TokaDB supports ByteStore and HDFS via a unified SDK that abstracts backend details and exposes a consistent interface. Both layers scale independently: compute nodes scale out horizontally without coupling to storage capacity, while the storage tier expands elastically without affecting compute placement. TokaDB deploys a **Cluster Manager** that provides a narrow control plane coordinating the compute tier. It tracks cluster metadata including membership, namespace, and partition placement while managing automatic failover to ensure reliability. The manager also performs load balancing based on partition utilization and enforces tenant isolation through authentication and role-based authorization. TokaDB supports **multiple access interfaces**. The *Table* API serves LRM workloads such as user behavior sequences and KV Cache data, while plugin-based *Vector* and *SQL* interfaces extend the system for vector retrieval and structured queries. TokaDB further integrates natively with modern data lakes by reading and writing *Parquet* tables directly, allowing external lake tables to be queried as native TokaDB tables and interoperating seamlessly with engines such as Apache Spark [51] and Flink [10] on shared storage.

6 EXPERIMENT

We evaluate the performance of the proposed TokaDB through comprehensive experiments covering three aspects. (1) **End-to-end evaluation:** benchmarking TokaDB against representative databases under real-world LRM workloads. (2) **Multi-dimensional performance analysis:** examining TokaDB's performance across five critical workload dimensions as shown in Figure 1(b). (3) **Design validation and sensitivity analysis:** isolating and quantifying the contributions of key system components.

6.1 Experiment Setting

Environment. All experiments are conducted on a dedicated cluster consisting of 6 compute nodes, each equipped with an Intel processor (128 logical cores), 512 GB of memory and 25GbE dual-port NICs (25 Gb/s, 2 ports per card). All nodes share a common storage subsystem comprising 24 SSDs (Samsung PM1733 NVMe, 7 TB each) and 72 HDDs (Seagate Exos X18, 18 TB each).

Workloads. We constructed two representative real-world production workloads at ByteDance: *E-commerce* and *Advertising* platforms. Table 2 summarizes the key metrics for both workloads. For each workload, the data comprise user behavior sequences from 10 million users and KV Cache data generated by 20 models. Compared with the *E-commerce* workload, the *Advertising* workload experiences substantially higher request traffic. As a result, users on the *Advertising* platform exhibit longer average behavior sequences and generate a much larger total volume of user behavior

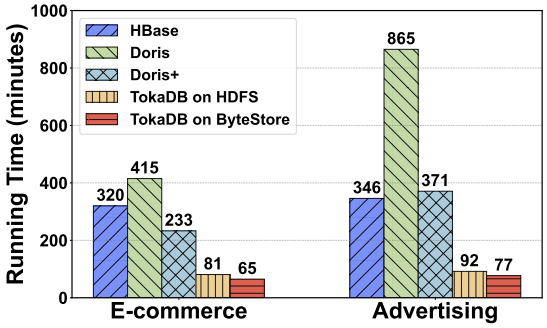


Figure 10: End-to-end running time under the *E-commerce* and *Advertising* LRM workloads.

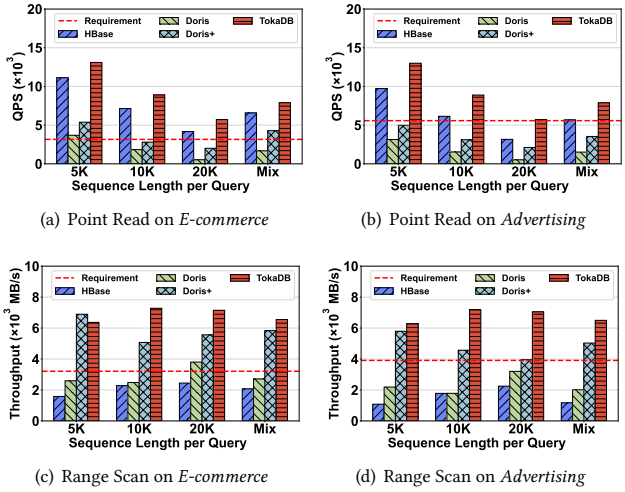
sequences. For KV Cache data, both workloads use the same model architectures, so each KV entry has the same size; however, the *Advertising* platform serves more active users, which leads to a significantly larger overall KV Cache dataset.

Competitors and Configurations. We compare TokaDB against two production-grade databases: HBase[1] and Doris[2]. Both systems are widely used in practice to manage LRM training-serving workloads within a single infrastructure. More recent systems such as Magnus [42] and Krypton [15] are not publicly available yet and support only a subset of the LRM workflow (§ 7), so we do not include them in our comparison. Both HBase and Doris adopt a compute-storage separation architecture and use HDFS as their storage backend. For Doris, we evaluate two configurations: the standard deployment, and Doris+, which adopts a column-optimized modeling strategy for user behavior sequences and is widely used within ByteDance. These two configurations share identical system settings and differ only in their data modeling approach. TokaDB is deployed on ByteStore, a high-throughput storage backend developed in-house at ByteDance. To reflect real-world production usage, our primary experiments run TokaDB on ByteStore. For fairness and to isolate storage-related effects, we also evaluate TokaDB on HDFS under identical compute configurations.

6.2 End-to-End Evaluation

We first evaluate the performance of the data engines under end-to-end LRM workloads. Each system is tested over a 100-minute execution window for both access patterns. Within this window, we measure the time required to complete an identical workload, which provides a unified basis for comparing overall performance. The total execution times for the two workloads are presented in Figure 10. The results show that TokaDB on ByteStore outperforms both HBase and Doris in all tested scenarios. Under the *E-commerce* workload, TokaDB on ByteStore achieves a 5× speedup over HBase and a 3× speedup over Doris. For the *Advertising* workload, the speedups reach 5× over HBase and 4× over Doris. TokaDB on HDFS does not benefit from ByteStore-specific optimizations. It is 31% slower than TokaDB on ByteStore in the *E-commerce* workload and 28% slower in the *Advertising* workload. Even so, it still achieves at least a 2.9× improvement over the other systems in both scenarios. It is worth noting that TokaDB is the only system that completed all

[1]https://hbase.apache.org
[2]https://doris.apache.org



(a) Point Read on *E-commerce* (b) Point Read on *Advertising*

(c) Range Scan on *E-commerce* (d) Range Scan on *Advertising*

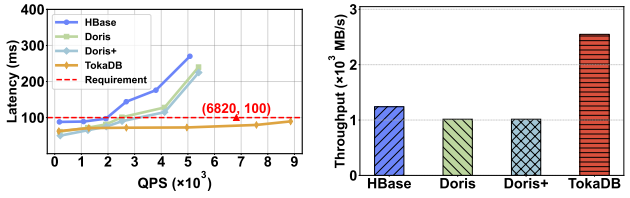
Figure 11: Point Read Throughput and Range Scan Bandwidth of the *E-commerce* and *Advertising* LRM workloads.

workloads within the 100-minute execution window. This indicates that, under our experimental resource constraints, the competitors cannot provide sufficient capacity to support LRM at a production scale. These gains mainly come from the proposed TokaDB unified storage engine (§4), whose components collectively provide high performance for both serving and training workloads in LRM.

6.3 Multi-Dimensional Performance Analysis

Point Read Throughput. We evaluate point read throughput under two workloads. Since LRM at different scales reads sequences of different lengths, we vary the sequence length per query to assess each system's ability to serve LRM across scales. We test four configurations: 5*K*, 10*K*, 20*K*, and *Mix*, where *Mix* replays a real production workload combining requests from multiple models. As shown in Figure 11(a) and (b), TokaDB consistently outperforms all baselines. Its performance advantage becomes more pronounced on the larger scale *Advertising* workload and on heavier configurations such as the 20*K* case. TokaDB reaches a 1.8× speedup over HBase, a 1.6× speedup over Doris, and a 1.3× speedup over Doris+ under the 10*K* configuration. It also maintains a 1.5× lead over HBase under the realistic *Mix* pattern. These results show that TokaDB efficiently handles long sequence processing by avoiding the join overhead present in Doris and Doris+. Its advantage over HBase is further attributed to the *Detached-Segment Index Layout* (§4.2), which, although originally designed for range scan, also reduces read amplification and improves point read performance.

Range Scan Bandwidth. This experiment uses the same configurations as the point read throughput test to evaluate range scan bandwidth under user behavior sequences workloads. Figure 11(c) and (d) show that TokaDB consistently outperforms both HBase and Doris, achieving up to 3.6× and 2.5× higher range scan bandwidth, respectively. Compared with Doris+, TokaDB achieves similar performance in low settings but scales noticeably better at larger configurations, reaching up to 1.7× improvements at the 20*K* level. This improvement is driven by the *Detached-Segment Index Layout* (§4.2), which is highly effective for range scan workloads. It avoids the join overhead in Doris+ and enables range scan operations to read only the time-filtered segments that are required, thereby maximizing I/O efficiency and sustaining high bandwidth.



(a) P99 Read Latency (b) Max Write Bandwidth

Figure 12: P99 read latency and max write bandwidth on KV Cache data.

Read Latency. We evaluate the P99 read latency of each system under KV Cache data workloads and vary the overall QPS to observe system behavior under increasing read pressure. Since KV Cache data characteristics are consistent across workloads, the maximum achievable performance is similar among them. As shown in Figure 12(a), TokaDB achieves both lower P99 read latency and higher sustainable QPS compared with HBase, Doris, and Doris+. TokaDB supports up to 30% more QPS while keeping P99 read latency below 100 ms across all tested load levels. Moreover, at comparable throughput, TokaDB reduces P99 read latency by as much as 70%. These improvements reflect the effectiveness of TokaDB's *Cost-Efficient Cooperative I/O Engine* (§4.3), which reduces long tail delays on the read path and ensures predictable performance.

Write Bandwidth. We measure the peak write bandwidth of TokaDB, HBase, Doris, and Doris+ under KV Cache data workloads. To make the comparison realistic, we set each system's read load to 85% of its own maximum QPS reported in Figure 12(a), and then measure the maximum sustainable write bandwidth under this steady read pressure. As shown in Figure 12(b), TokaDB achieves the highest write bandwidth among the evaluated systems, outperforming HBase by 52% and Doris and Doris+ by 60%. These results demonstrate the effectiveness of TokaDB's *Cost-Efficient Cooperative I/O Engine* (§4.3) in maintaining high write bandwidth.

Cost. We evaluate the total cost required by each system to serve the full real-world LRM workload. For each system, we provide compute and storage resources until it can just sustain the target LRM workload within the required operational time window. Since HBase, Doris and Doris+ cannot meet the P99 read latency requirement of 100 ms even with scaled-out hardware, we relax their target to 160 ms for the purpose of this cost comparison. Based on the hardware counts presented in Table 3, we compute the total daily cost for each system using the public selling prices of VolcanoEngine [9]. The prices used in our calculation are 50.29 USD/day for a 64-core and 512-GB compute node, 5.75 USD/day for a 7 TB SSD, and 2.99 USD/day for a 20 TB HDD.

The results show that TokaDB achieves 2× to 3× greater cost-effectiveness than baselines for LRM workloads, offering a superior cost-performance balance. It reduces compute nodes and SSDs via the *Hybrid-Access Specialized Data Organization* and *Priority-Based Resource Manager*, lowering CPU consumption and enhancing

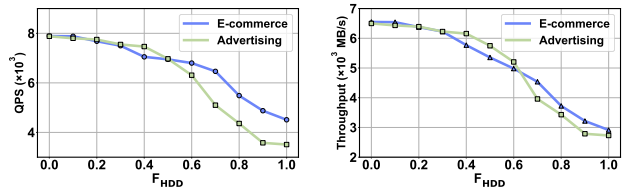
Table 3: Cost comparison between TokaDB and competitors.

(a) Cost comparison on *E-commerce* workload.

System	Compute Nodes	SSDs	HDDs	Total Cost (USD/day)
HBase	12	38	197	1411.01
Doris	18	37	207	1736.9
Doris+	8	24	167	1039.65
TokaDB	6	24	72	655.02

(b) Cost comparison on *Advertising* workload.

System	Compute Nodes	SSDs	HDDs	Total Cost (USD/day)
HBase	17	49	314	2075.54
Doris	29	54	302	2671.89
Doris+	12	24	262	1524.86
TokaDB	6	24	72	655.02



(a) Point Read Throughput (b) Range Scan Bandwidth

Figure 13: Parameter sensitivity for Hot-Cold Tiering.

utilization. Meanwhile, the *Cost-Efficient Cooperative I/O Engine* optimizes HDD usage, requiring only 72 HDDs versus 167–314 for others. It is particularly noteworthy that TokaDB maintained an identical hardware configuration across both the *E-commerce* and *Advertising* workloads, since it does not reach its system bottleneck and retains sufficient remaining resource capacity.

6.4 Design Validation and Sensitivity Analysis

Locality-Triggered Garbage Collection (LTGC). We evaluate LTGC under LRM workloads after ingestion completes. For comparison, we use as the baseline the same GC mechanism as BlobDB [22], an open-source and widely used KV-separation [55] variant of RocksDB. LTGC is enabled using the trigger in Eq. 1, which is controlled by an average locality threshold θ and a high-locality threshold τ. We sweep the parameter pairs $(\theta, \tau) \in \{(2,4),(3,5),(4,6)\}$ while keeping γ fixed. The hardware configuration, compaction concurrency, and ingestion rate are identical across all runs, and the *Detached-Segment Index Layout* is enabled throughout the evaluation. We report three metrics: the *GC I/O*, the average locality ($\bar{LS}$), and the high-locality ratio (F_{LS}), as shown in Table 4. Across both workloads, LTGC reduces GC and compaction I/O while improving locality compared with BlobDB GC. It cuts total I/O by up to 78% (from 266.8 TB to 58.6 TB) and lowers locality dispersion by about 79% (average locality from 8.2 to 1.7). Lower thresholds such as $\theta = 2$ and $\tau = 4$ trigger GC more aggressively and achieve the strongest locality at moderate I/O cost. Mid-range thresholds such as $\theta = 3$ and $\tau = 5$ provide the best overall balance between I/O reduction and locality improvement. Higher thresholds such as $\theta = 4$ and $\tau = 6$ further reduce I/O with a small decrease in locality tightening.

Table 4: Locality-Triggered GC performance metrics.

(a) Performance on *E-commerce* workload.

GC Method	GC IO	Avg locality	High-locality Ratio
BlobDB GC	170.8 TB	6.2	11
LTGC θ=2, τ=4	122.2 TB	2.1	3
LTGC θ=3, τ=5	77.3 TB	2.7	6
LTGC θ=4, τ=6	48.6 TB	5.1	7

(b) Performance on *Advertising* workload.

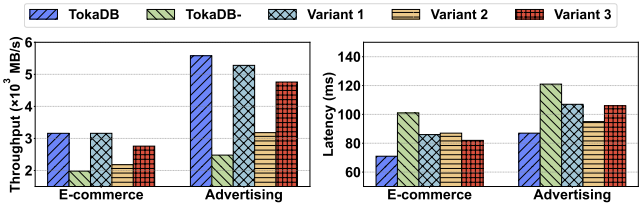
GC Method	GC IO	Avg locality	High-locality Ratio
BlobDB GC	266.8 TB	8.2	13
LTGC θ=2, τ=4	163.2 TB	1.7	4
LTGC θ=3, τ=5	87.3 TB	2.8	6
LTGC θ=4, τ=6	58.6 TB	5.9	7

Hot-Cold Tiering Parameter Sensitivity. We evaluate how the hot-cold tiering configuration influences performance and storage efficiency. The key parameter under examination is F_{HDD}, which specifies the planned fraction of each user's data that is placed on the HDD tier. For each value of F_{HDD}, we increase the system load to its highest sustainable level and measure the resulting point read throughput in QPS and range scan bandwidth in MB/s. As shown in Figure 13, both workloads show similar overall trends. Point read throughput remains stable at low F_{HDD} values and begins to decline once the HDD fraction exceeds about 0.3, with a pronounced drop beyond F_{HDD}>0.7. For the *Advertising* workload, it exhibits slightly better tolerance to higher HDD ratios due to its larger proportion of cold data. Range scan bandwidth follows the same pattern-remaining steady at small F_{HDD}, gradually falling in the mid-range, and dropping sharply once HDD dominates-reflecting the increasing access latency to cold data blocks. Overall, TokaDB sustains near-peak performance even when up to 40% of data resides on the HDD tier, providing a tunable configuration that enables flexible performance-cost trade-offs.

Ablation of Cost-Efficient Cooperative I/O Engine. To understand the contribution of each optimization within the cost-efficient cooperative I/O engine (§ 4.3), we conduct an ablation analysis by selectively disabling individual components and comparing the results with the fully optimized baseline. In Table 5, each row labeled "–" indicates that a specific optimization is disabled, while the row labeled "all methods enabled" represents the full configuration. The experimental results show that each optimization reduces tail latency. *Advertising* workload issues a higher request rate than the *E-commerce* workload, yet the trends are consistent across both. Disabling *Adaptive Backup Reads* causes the largest degradation: P99 rises from 71 ms to 117 ms on *E-commerce* workload and from 87 ms to 132 ms on *Advertising*. This indicates that backup reads are essential for masking slow replicas, which are common in our cluster. *Hardware-Aware Block Alignment* also matters: P99 increases to 94 ms and 113 ms for the two workloads, confirming that aligning I/O with physical block boundaries reduces random seeks. Although *Log as Data* targets writes, disabling it still increases read P99 (71 ms to 89 ms; 87 ms to 124 ms). The effect is stronger on *Advertising* because its higher traffic makes reads and writes contend for the same HDD bandwidth; lowering write amplification frees more bandwidth for reads and thus reduces tail latency.

Table 5: Ablation of Cost-Efficient Cooperative I/O Engine.

TokaDB Variant	P99 Read Latency	
	E-commerce	Advertising
With All Methods	71ms	87ms
- Adaptive Backup Reads	117ms	132ms
- Hardware-Aware Block Alignment	94ms	113ms
- Log as Data	89ms	124ms



(a) Impact on Point Read Throughput (b) Impact on P99 Read Latency

Figure 14: Ablation of Priority-Based Resource Manager.

Ablation of Priority-Based Resource Manager. We evaluate the contribution of each strategy in the *Priority-Based Resource Manager* using five variants of TokaDB. TokaDB enables all resource management components, while TokaDB- disables the entire module and serves as the baseline. *Variant 1*, *Variant 2*, and *Variant 3* each remove one component, namely the *Preemptive Thread Scheduler*, the *Hierarchical Cache Allocator*, and the *Multi-Layer Integrated Rate Limiter*. We evaluate these variants using two metrics: point read throughput and P99 read latency, under two LRM workloads to quantify their impact. As shown in Figure 14, disabling the *Preemptive Thread Scheduler* (*Variant 1*) has little effect on point read throughput, but it significantly increases P99 read latency because high-priority tasks can no longer preempt low-priority operations. Removing the *Hierarchical Cache Allocator* (*Variant 2*) causes a clear drop in point read throughput and increases P99 read latency. Without cache isolation, index blocks and other critical data are more easily evicted, forcing queries to traverse longer read paths and incur additional storage accesses. Turning off the *Multi-Layer Integrated Rate Limiter* (*Variant 3*) simultaneously reduces point read throughput and raises P99 read latency, since low-priority workloads consume I/O bandwidth aggressively and interfere with latency-sensitive operations. Across the two workloads, these effects are more pronounced in *Advertising* due to its higher load and stronger resource contention. Under increased system pressure, cache isolation and I/O throttling become crucial for sustaining throughput, while CPU preemption is essential for maintaining low tail latency. The lighter *E-commerce* workload exhibits more stable behavior, but the same performance patterns remain consistent.

7 RELATED WORK

Despite advanced feature design and access to massive user behavior datasets, most real-world deep learning recommendation models (DLRM) [16, 56, 58] systems still scale poorly in computation [59]. Motivated by the strong generalization ability and rich world knowledge of large language models (LLMs) [17], large recommendation models (LRM) have been widely deployed in industry [11, 14, 26, 49, 52]. HSTU [52] explores scaling industrial LRMs into the trillion-parameter regime. LONGER [11] and HLLM [14] explore improved sequence modeling and cold-start handling within LRM architectures. Complementary to these LRM advances, TokaDB focuses on the data storage layer to enable large-scale industrial deployment of LRM at ByteDance.

Recent work has started to optimize data storage for LRM. At the **cache layer**, MARM [35] augments LRM with large recommendation caches to exploit cache scaling laws at production scale, while ReCXL [31] uses near CXL [18] memory processing for efficient embedding training with reduced data movement. At the **storage layer**, systems such as Krypton [15] and Magnus [42] support only a subset of LRM workloads in current deployments. Krypton [15] demonstrates performance on hybrid serving/analytical workloads but lacks support for LRM training data management. Magnus [42], built on Apache Iceberg [3], optimizes data management for LRM training over large-scale feature data but does not support low-latency, high-throughput serving workloads required for online inference. Open-source databases such as HBase [47] and Doris [4] have mature ecosystems, proven scalability, and are widely deployed in large data platforms. They have recently been adopted as the storage backend for emerging industrial LRM pipelines in several practical settings but still lack LRM-specific optimizations (discussed in §3). To bridge this gap, TokaDB is a unified storage engine supporting both LRM training and serving.

8 CONCLUSION

In this work, we present TokaDB, a unified storage engine for managing both user behavior sequences and KV Cache data across the training and serving stages of large recommendation models (LRM). Through a *Hybrid-Access Data Organization*, a *Cost-Efficient Cooperative I/O Engine*, and a *Priority-Based Resource Manager*, TokaDB tackles the core challenges of unifying the management of user behavior sequences and KV Cache data across both training and serving stages. TokaDB has been deployed in production and serves as the data infrastructure for real LRM workloads at ByteDance. Extensive experiments show TokaDB outperforms HBase, Doris and Doris+ by 4–10× and meets all LRM multi-dimensional requirements. We believe TokaDB provides a practical foundation for large recommendation systems and opens opportunities for cross-modal unification and workload-aware optimization.

Acknowledgments

This work was supported in part by National Key R&D Program of China (No. 2023YFB4502801), National Natural Science Foundation of China (Nos. 62402187 and U22A2027), China Postdoctoral Science Foundation (Nos. GZB20240243 and 2024M751009), and Postdoctoral Project of Hubei Province (No. 2024HBBHCXA024). We thank Lei Zhang, Xian Liu, Shuai Zhang, Chang Liu, Lei Li, Jianfeng Zhou, Liang Chen, Jianqiu Zhang, Dongsheng Zhao, Kang Li, Jie Yang, and all members of the TokaDB team for their valuable contributions to this work. We also thank Chaofei Xu, Jinxin Liu, and Xu Cheng from the ByteStore team for their support on the underlying storage infrastructure. We are grateful to Jianshun Zhang for his valuable academic support.

References

[1] Daniel J Abadi, Peter A Boncz, and Stavros Harizopoulos. 2009. Column-oriented database systems. *Proceedings of the VLDB Endowment* 2, 2 (2009), 1664–1665.

[2] Daniel J Abadi, Samuel R Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: how different are they really?. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 967–980.

[3] Apache Iceberg. [n. d.]. Apache Iceberg: The Open Table Format for Analytic Datasets. https://iceberg.apache.org/. Accessed: 2025-11-15.

[4] Apache Software Foundation. 2023. Apache Doris: High-performance real-time analytical database. https://doris.apache.org/. Accessed: 2025-10-30.

[5] Newsha Ardalani, Carole-Jean Wu, Zeliang Chen, Bhargav Bhushanam, and Adnan Aziz. 2022. Understanding scaling laws for recommendation models. *arXiv preprint arXiv:2208.08489* (2022).

[6] Joy Arulraj, Andrew Pavlo, and Prashanth Menon. 2016. Bridging the archipelago between row-stores and column-stores for hybrid workloads. In *Proceedings of the 2016 International Conference on Management of Data*. 583–598.

[7] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. 2024. Reducing transformer key-value cache size with cross-layer attention. *Advances in Neural Information Processing Systems* 37 (2024), 86927–86957.

[8] ByteDance. 2025. TerarkDB. Available online. https://github.com/bytedance/terarkdb

[9] ByteDance. 2025. VolcanoEngine. Available online. https://www.volcengine.com

[10] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* 38, 4 (2015).

[11] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. Longer: Scaling up long sequence modeling in industrial recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 247–256.

[12] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, et al. 2023. TWIN: TWo-stage interest network for lifelong user behavior modeling in CTR prediction at kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3785–3794.

[13] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[14] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. Hllm: Enhancing sequential recommendations via hierarchical large language models for item and user modeling. *arXiv preprint arXiv:2409.12740* (2024).

[15] Jianjun Chen, Rui Shi, Heng Chen, Li Zhang, Ruidong Li, Wei Ding, Liya Fan, Hao Wang, Mu Xiong, Yuxiang Chen, et al. 2023. Krypton: real-time serving and analytical SQL engine at ByteDance. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3528–3542.

[16] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

[17] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. M6-rec: Generative pretrained language models are open-ended recommender systems. *arXiv preprint arXiv:2205.08084* (2022).

[18] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An introduction to the compute express link (cxl) interconnect. *Comput. Surveys* 56, 11 (2024), 1–37.

[19] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).

[20] Peter J Denning. 1968. The working set model for program behavior. *Commun. ACM* 11, 5 (1968), 323–333.

[21] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Transactions on Storage (TOS)* 17, 4 (2021), 1–32.

[22] Facebook. 2018. BlobDB. http://rocksdb.org/. Accessed: October 3, 2021.

[23] Chen Gao, Zixin Zhao, Lv Shao, and Tong Liu. 2025. Next Interest Flow: A Generative Pre-training Paradigm for Recommender Systems by Modeling All-domain Movelines. *arXiv preprint arXiv:2510.11317* (2025).

[24] Zhicheng He, Weiwen Liu, Wei Guo, Jiarui Qin, Yingxue Zhang, Yaochen Hu, and Ruiming Tang. 2023. A survey on user behavior modeling in recommender systems. *arXiv preprint arXiv:2302.11087* (2023).

[25] Kaisong Huang, Tianzheng Wang, Qingqing Zhou, and Qingzhong Meng. 2023. The art of latency hiding in modern database engines. *Proceedings of the VLDB Endowment* 17, 3 (2023), 577–590.

[26] Jian Jia, Yipei Wang, Yan Li, Honggang Chen, Xuehan Bai, Zhaocheng Liu, Jian Liang, Quan Chen, Han Li, Peng Jiang, et al. 2025. LEARN: Knowledge Adaptation from Large Language Model to Recommendation for Practical Industrial Application. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 11861–11869.

[27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[28] Weijiang Lai, Beihong Jin, Yapeng Zhang, Yiyuan Zheng, Rui Zhao, Jian Dong, Jun Lei, and Xingxing Wang. 2025. Modeling Long-term User Behaviors with Diffusion-driven Multi-interest Network for CTR Prediction. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 289–298.

[29] Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *ACM SIGOPS operating systems review* 44, 2 (2010), 35–40.

[30] Jiayi Liao, Ruobing Xie, Sihang Li, Xiang Wang, Xingwu Sun, Zhanhui Kang, and Xiangnan He. 2025. Multi-Grained Patch Training for Efficient LLM-based Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1572–1581.

[31] Haifeng Liu, Long Zheng, Yu Huang, Jingyi Zhou, Chaoqiang Liu, Runze Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. 2024. Enabling efficient large recommendation model training with near cxl memory processing. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 382–395.

[32] Hui Lu, Zheng Chai, Yuchao Zheng, Zhe Chen, Deping Xie, F Peng Xu, Xun Zhou, and Di Wu. 2025. Large Memory Network for Recommendation. In *Companion Proceedings of the ACM on Web Conference 2025*. 1162–1166.

[33] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Hariharan Gopalakrishnan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2017. Wisckey: Separating keys from values in ssd-conscious storage. *ACM Transactions On Storage (TOS)* 13, 1 (2017), 1–28.

[34] Ruiming Lu, Erci Xu, Yiming Zhang, Fengyi Zhu, Zhaosheng Zhu, Mengtian Wang, Zongpeng Zhu, Guangtao Xue, Jiwu Shu, Minglu Li, et al. 2023. Perseus: A {Fail-Slow} detection framework for cloud storage systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 49–64.

[35] Xiao Lv, Jiangxia Cao, Shijie Guan, Xiaoyou Zhou, Zhiguang Qi, Yaqiang Zang, Ming Li, Ben Wang, Kun Gai, and Guorui Zhou. 2024. MARM: Unlocking the Future of Recommendation Systems through Memory Augmentation and Scalable Complexity. *arXiv preprint arXiv:2411.09425* (2024).

[36] Richard L. Mattson, Jan Gecsei, Donald R. Slutz, and Irving L. Traiger. 1970. Evaluation techniques for storage hierarchies. *IBM Systems journal* 9, 2 (1970), 78–117.

[37] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta informatica* 33, 4 (1996), 351–385.

[38] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.

[39] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of machine learning and systems* 5 (2023), 606–624.

[40] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. Clickhouse-lightning fast analytics for everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744.

[41] Tianhao Shi, Yang Zhang, Zhijian Xu, Chong Chen, Fuli Feng, Xiangnan He, and Qi Tian. 2024. Preliminary study on incremental learning for large language model-based recommender systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4051–4055.

[42] Jun Song, Jingyi Ding, Irshad Kandy, Yanghao Lin, Zhongjia Wei, Zilong Zhou, Zhiwei Peng, Jixi Shan, Hongyue Mao, Xiuqi Huang, et al. 2025. Magnus: A Holistic Approach to Data Management for Large-Scale Machine Learning Workloads. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4964–4977.

[43] Mike Stonebraker, Daniel J Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O'Neil, et al. 2018. C-store: a column-oriented DBMS. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*. 491–518.

[44] Yijia Sun, Shanshan Huang, Zhiyuan Guan, Qiang Luo, Ruiming Tang, Kun Gai, and Guorui Zhou. 2025. GRank: Towards Target-Aware and Streamlined Industrial Retrieval with a Generate-Rank Framework. *arXiv preprint arXiv:2510.15299* (2025).

[45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[46] Deepak Vohra. 2016. Apache parquet. In *Practical Hadoop Ecosystem: A Definitive Guide to Hadoop-Related Frameworks and Tools*. Springer, 325–335.

[47] Mehul Nalin Vora. 2011. Hadoop-HBase for large-scale data. In *Proceedings of 2011 International Conference on Computer Science and Network Technology*, Vol. 1. IEEE, 601–605.

[48] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[49] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 12–22.

[50] Hyunsik Yoo, Ting-Wei Li, SeongKu Kang, Zhining Liu, Charlie Xu, Qilin Qi, and Hanghang Tong. 2025. Continual Low-Rank Adapters for LLM-based Generative Recommender Systems. *arXiv preprint arXiv:2510.25093* (2025).

[51] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster computing with working sets. In *2nd USENIX workshop on hot topics in cloud computing (HotCloud 10)*.

[52] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[53] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. 2024. Wukong: Towards a scaling law for large-scale recommendation. *arXiv preprint arXiv:2403.02545* (2024).

[54] Gaowei Zhang, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Scaling law of large sequential recommendation models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 444–453.

[55] Jianshun Zhang, Fang Wang, Sheng Qiu, Yi Wang, Jiaxin Ou, Junxun Huang, Baoquan Li, Peng Fang, and Dan Feng. 2024. Scavenger: Better space-time trade-offs for key-value separated LSM-trees. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4072–4085.

[56] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* 52, 1 (2019), 1–38.

[57] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).

[58] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. 2019. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM conference on recommender systems*. 43–51.

[59] Zhuokai Zhao, Yang Yang, Wenyu Wang, Chihuang Liu, Yu Shi, Wenjie Hu, Haotian Zhang, and Shuang Yang. 2023. Breaking the curse of quality saturation with user-centric ranking. *arXiv preprint arXiv:2305.15333* (2023).

[60] Xuanhe Zhou, Wei Zhou, Liguo Qi, Hao Zhang, Dihao Chen, Bingsheng He, Mian Lu, Guoliang Li, Fan Wu, and Yuqiang Chen. 2025. OpenMLDB: A Real-Time Relational Data Feature Computation System for Online ML. In *Companion of the 2025 International Conference on Management of Data*. 729–742.

[61] Pablo Zivic, Hernan Vazquez, and Jorge Sánchez. 2024. Scaling sequential recommendation models with transformers. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1567–1577.